Internet Control Message Protocol (ICMP)

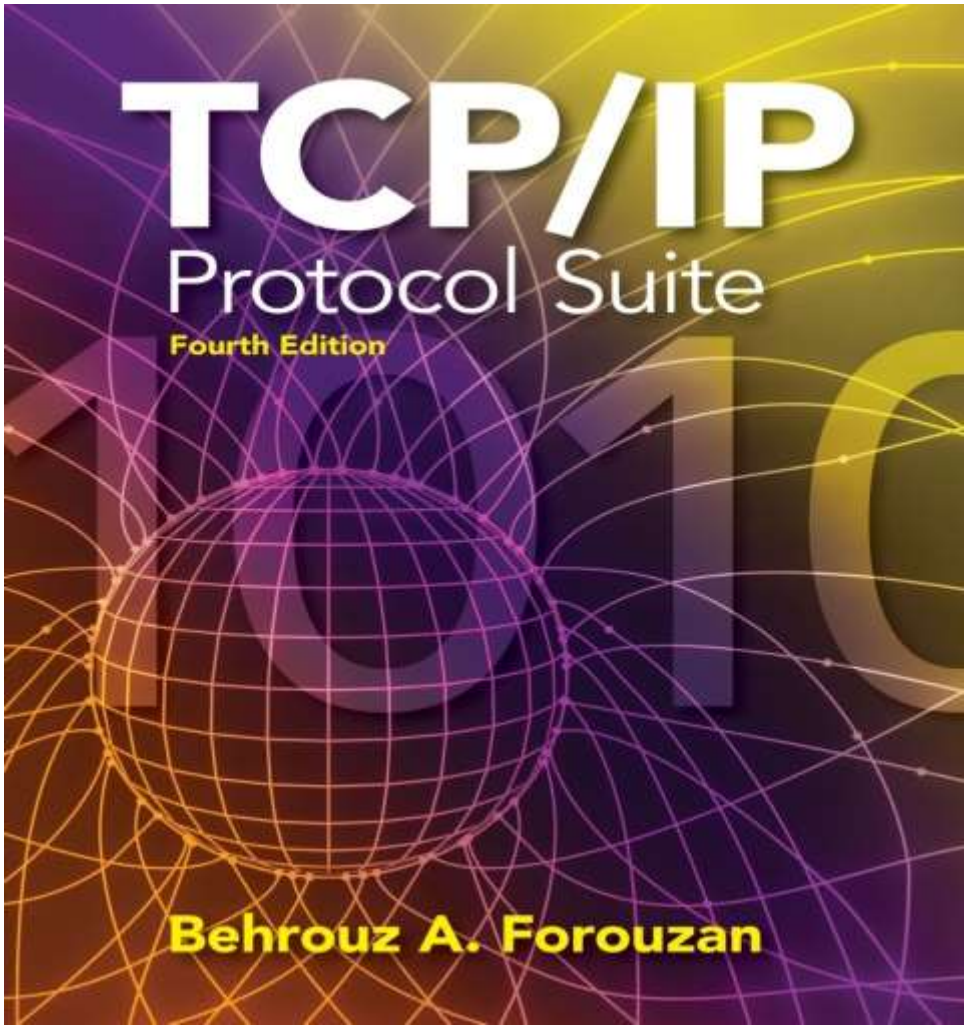
Objectives

Upon completion you will be able to:

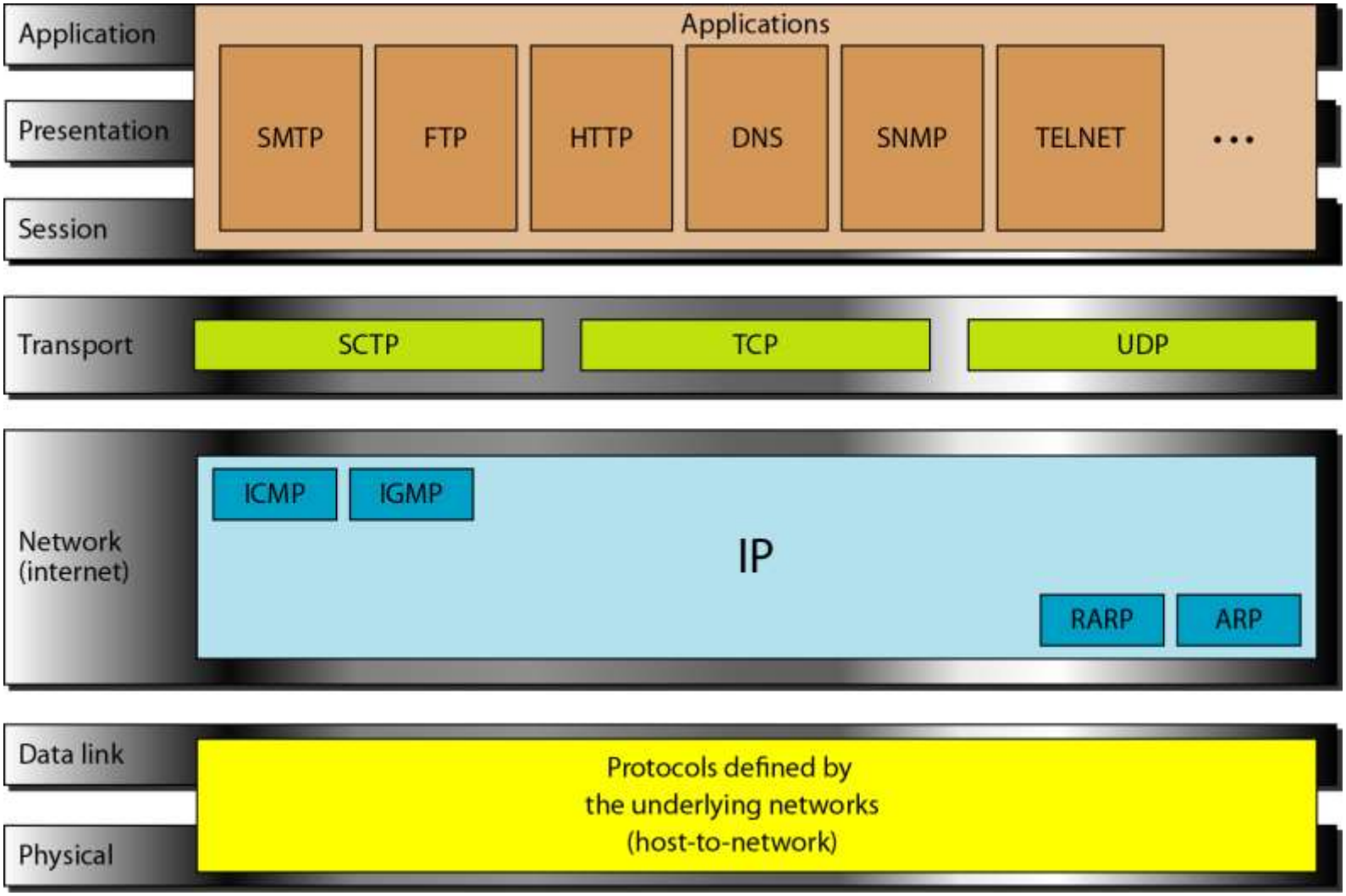
- *Be familiar with the ICMP message format*
- *Know the types of error reporting messages*
- *Know the types of query messages*
- *Be able to calculate the ICMP checksum*
- *Know how to use the ping and traceroute commands*
- *Understand the modules and interactions of an ICMP package*

Reference

- Chapter 9

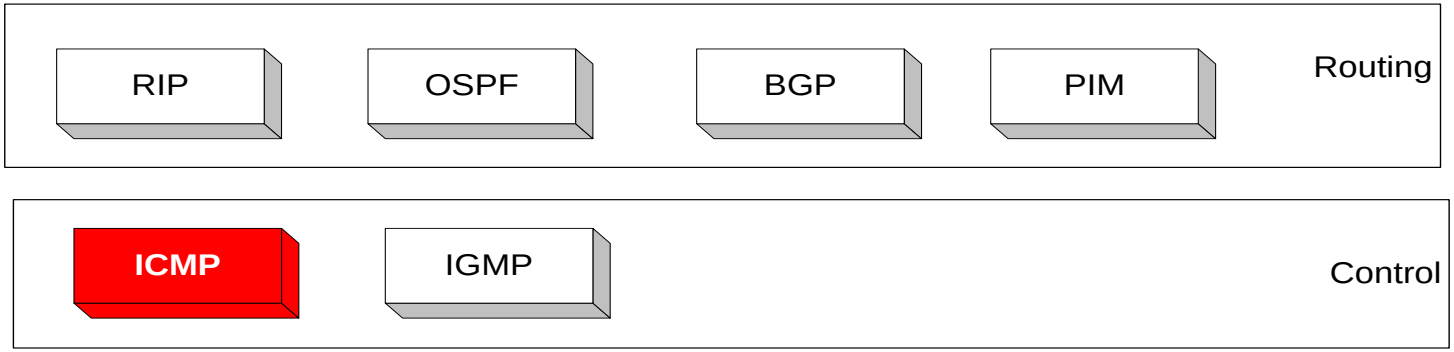


TCP/IP and OSI Model



Internet Protocol (IP)

- The IP (Internet Protocol) relies on several other protocols to perform necessary control and routing functions:
  - Control functions (ICMP)
  - Multicast signaling (IGMP)
  - Setting up routing tables (RIP, OSPF, BGP, PIM, …)



IP and Errors Handling

The IP protocol has no error-reporting or error correcting mechanism. What happens if something goes wrong? What happens if a router must discard a datagram because it cannot find a router to the final destination, or because the time-to-live field has a zero value? These are examples of situations where an error has occurred and the IP protocol has no built-in mechanism to notify the original host.

Figure 9.1 *Position of ICMP in the network layer*

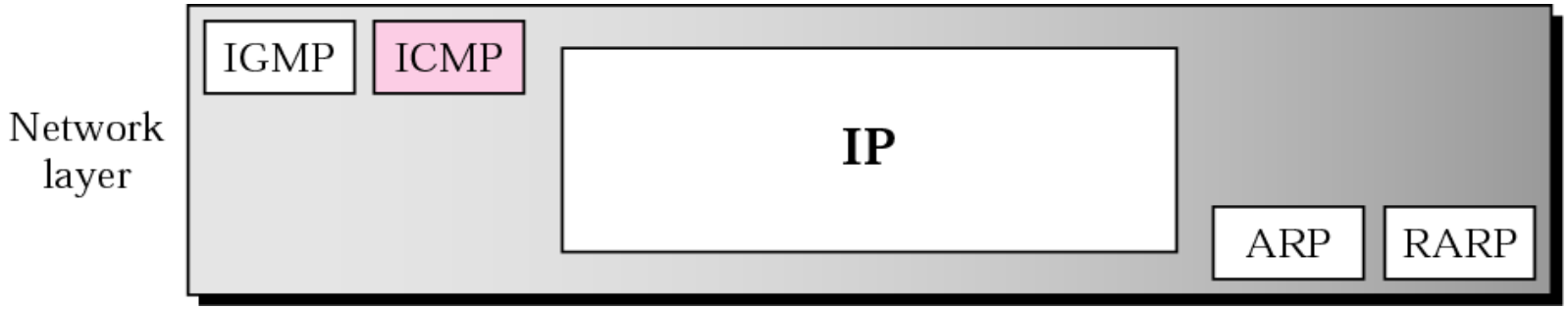
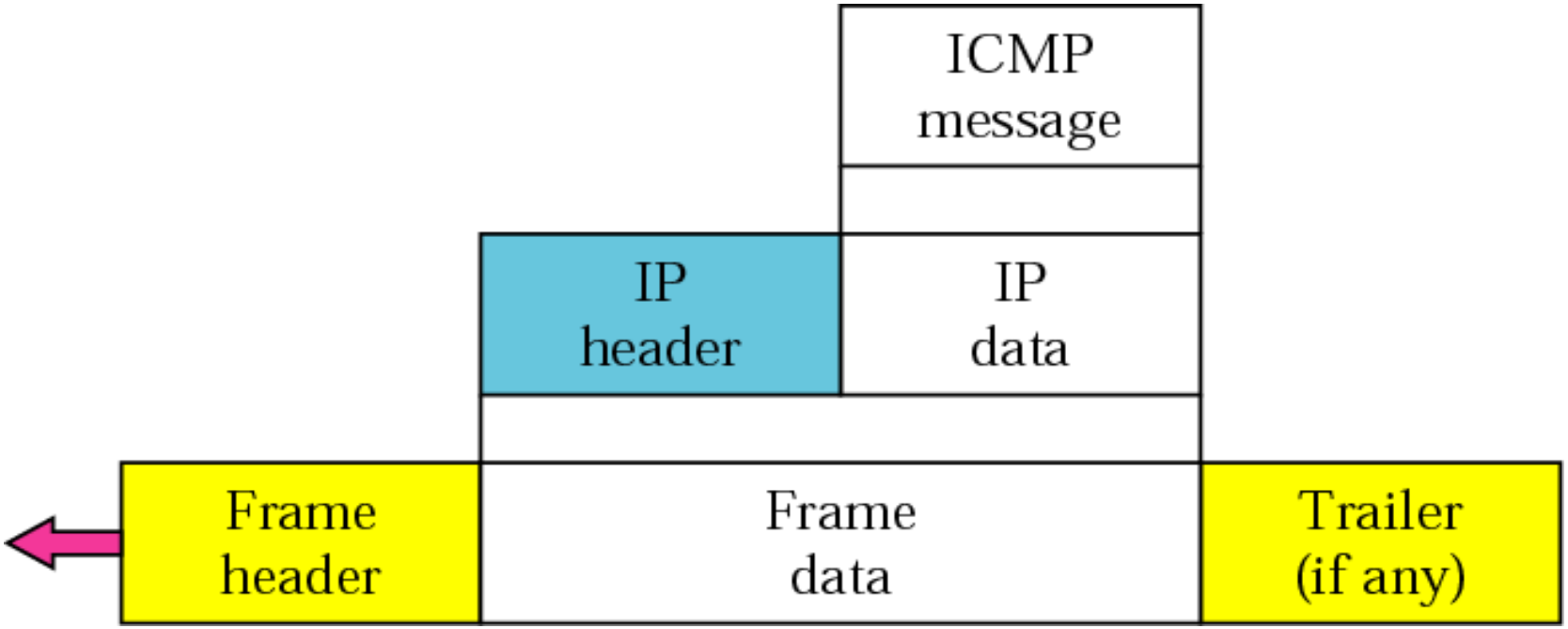


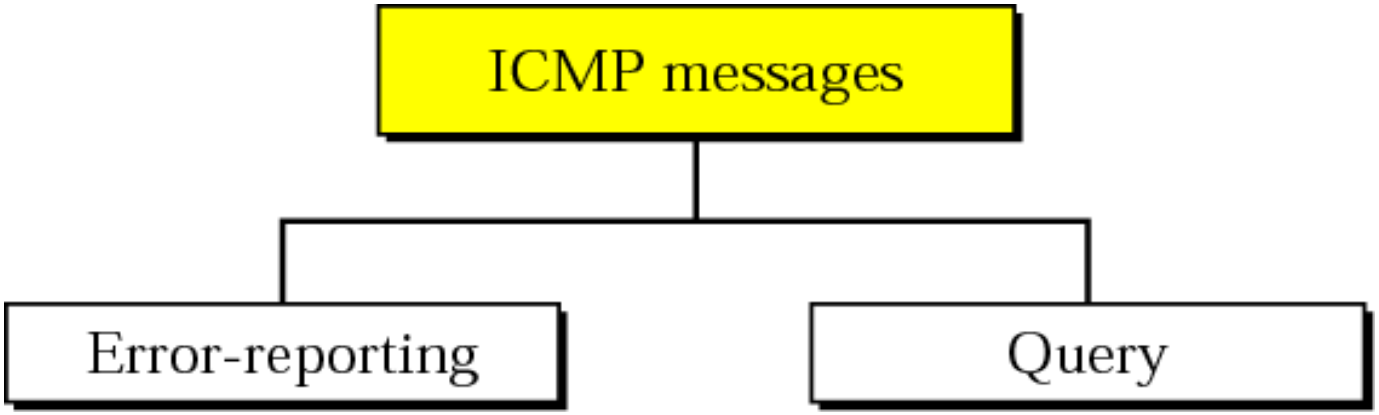
Figure 9.2 *ICMP encapsulation*



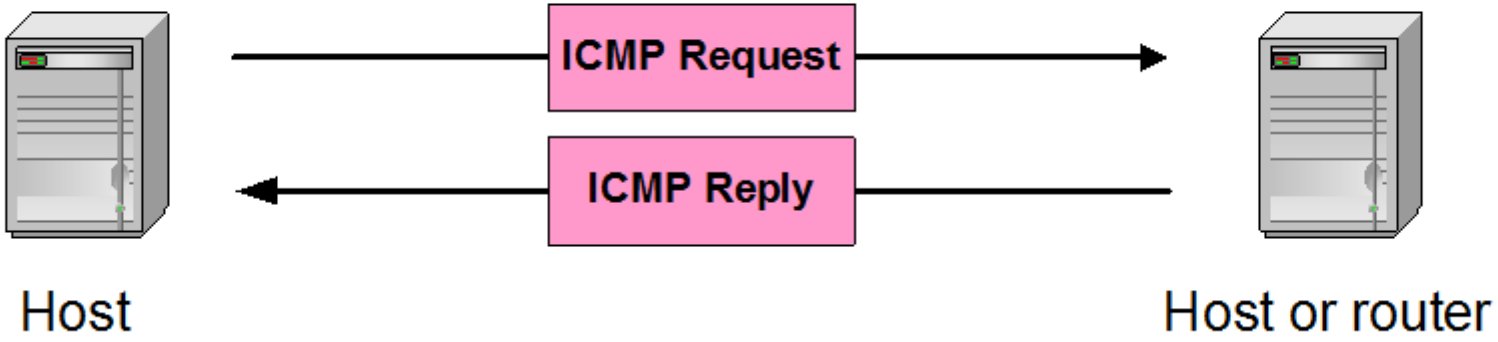
9.1 TYPES OF MESSAGES

ICMP messages are divided into error-reporting messages and query messages. The error-reporting messages report problems that a router or a host (destination) may encounter. The query messages get specific information from a router or another host.

Figure 9.3 *ICMP messages*



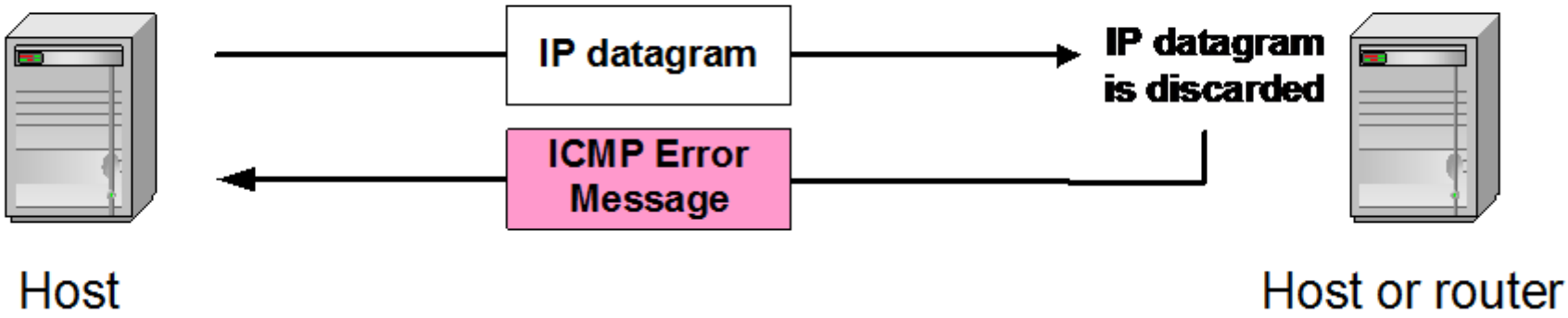
ICMP Query message



ICMP query:

- **Request sent by host to a router or host**
- **Reply sent back to querying host**

ICMP Error message



- **ICMP error messages report error conditions**
- **Typically sent when a datagram is discarded**
- **Error message is often passed from ICMP to the application program**

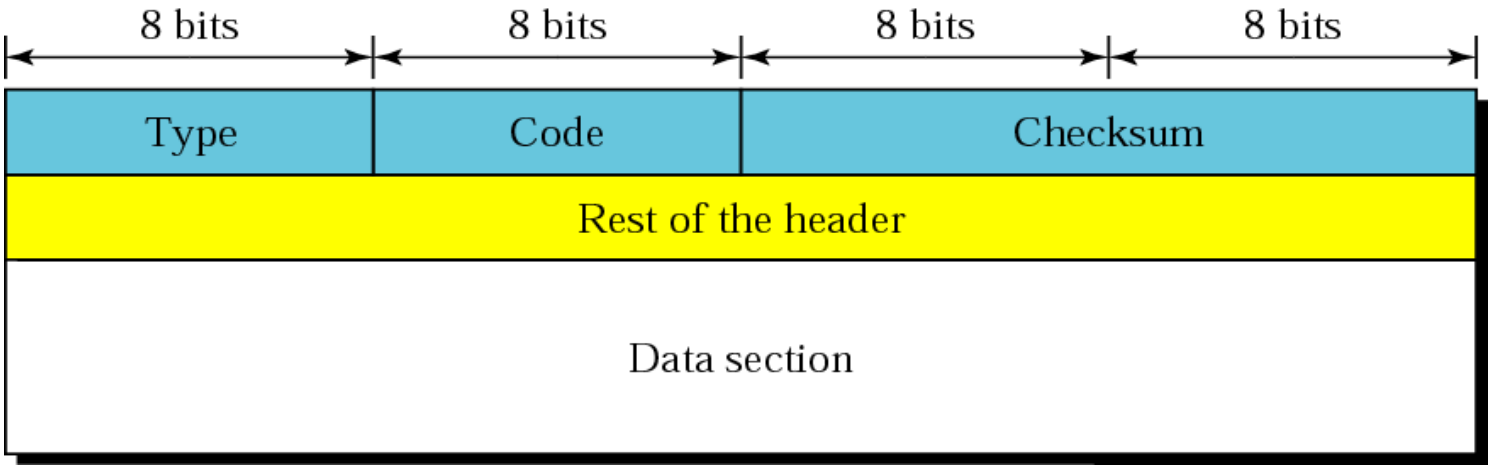
Table 9.1 ICMP messages

Category	*Type*	*Message*
Error-reporting messages	3	Destination unreachable
	4	Source quench
	11	Time exceeded
	12	Parameter problem
	5	Redirection
Query messages	8 or 0	Echo request or reply
	13 or 14	Timestamp request or reply
	17 or 18	Address mask request or reply
	10 or 9	Router solicitation or advertisement

9.2 MESSAGE FORMAT

An ICMP message has an 8-byte header and a variable-size data section. Although the general format of the header is different for each message type, the first 4 bytes are common to all.

Figure 9.4 *General format of ICMP messages*



9.3 ERROR REPORTING

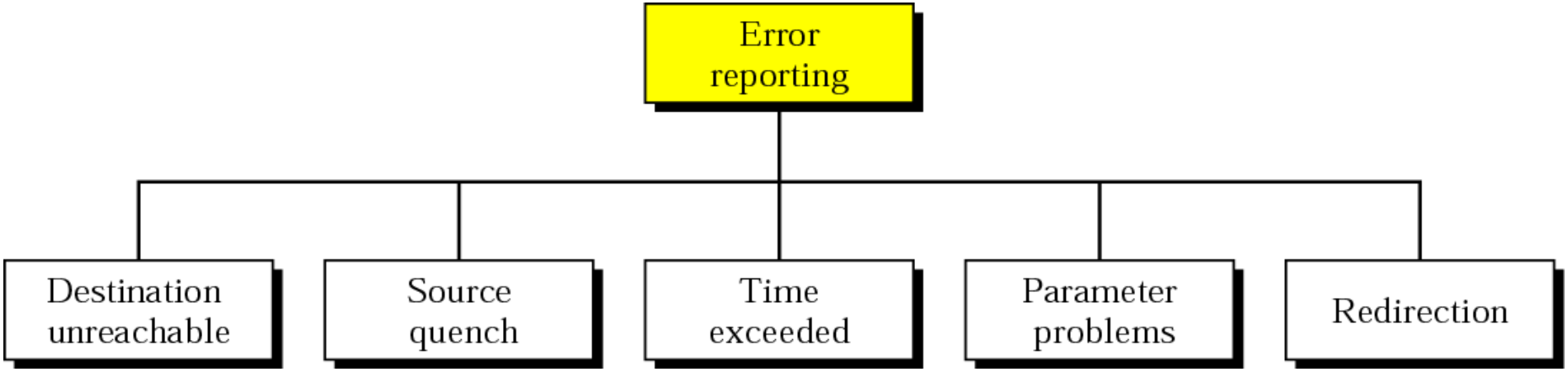
IP, as an unreliable protocol, is not concerned with error checking and error control. ICMP was designed, in part, to compensate for this shortcoming. ICMP does not correct errors, it simply reports them.

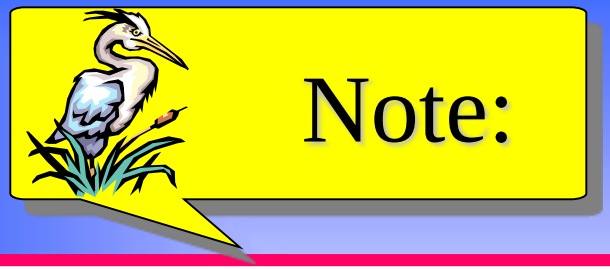
The topics discussed in this section include:

Destination Unreachable
Source Quench
Time Exceeded
Parameter Problem
Redirection

ICMP always reports error messages to the original source.

Figure 9.5 *Error-reporting messages*





Note:

The following are important points about ICMP error messages:

- *No ICMP error message will be generated in response to a datagram carrying an ICMP error message.*
- *No ICMP error message will be generated for a fragmented datagram that is not the first fragment.*
- *No ICMP error message will be generated for a datagram having a multicast address.*
- *No ICMP error message will be generated for a datagram having a special address such as 127.0.0.0 or 0.0.0.0.*

Figure 9.6 *Contents of data field for the error messages*

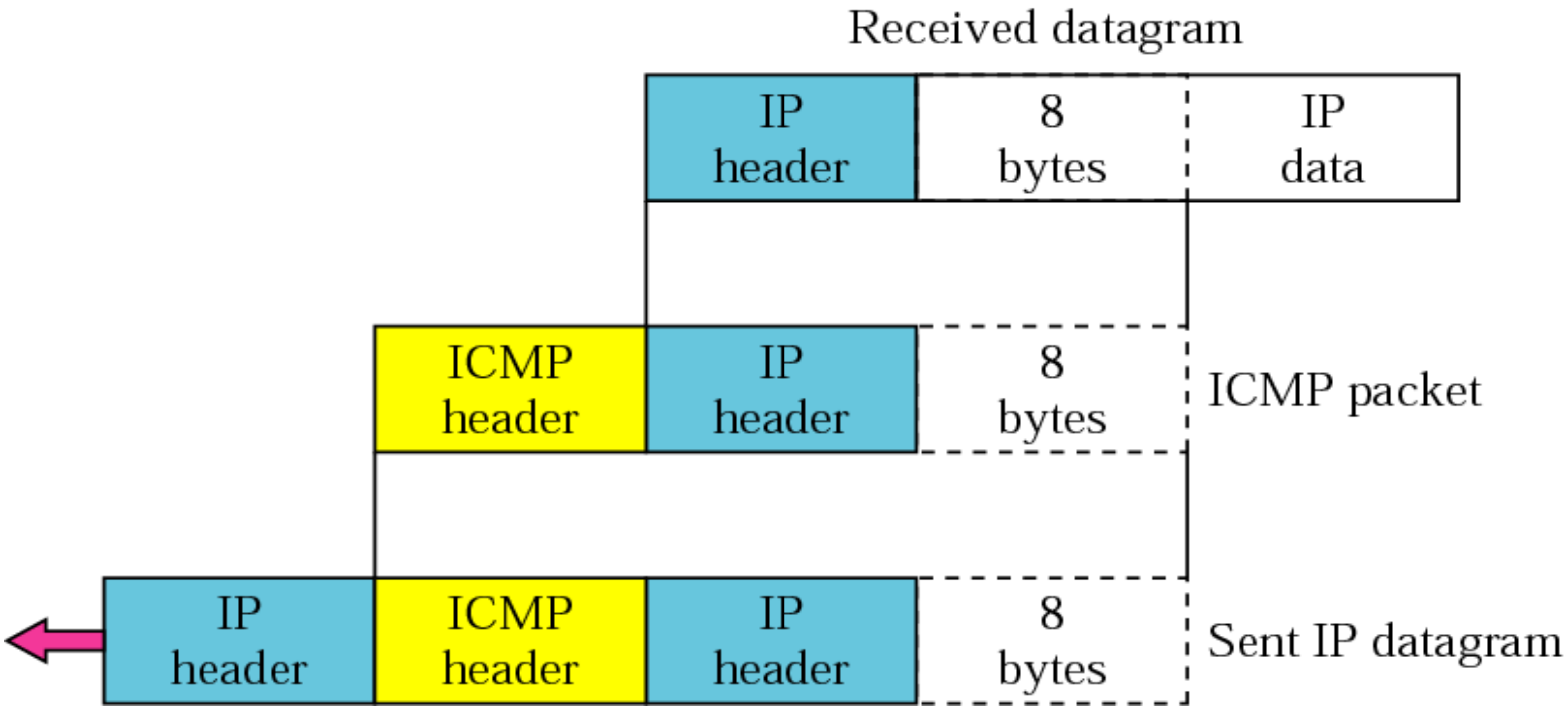


Figure 9.7 *Destination-unreachable format*

Type: 3	Code: 0 to 15	Checksum
Unused (All 0s)		
Part of the received IP datagram including IP header plus the first 8 bytes of datagram data		

Destination-unreachable messages with codes 2 or 3 can be created only by the destination host.

Other destination-unreachable messages can be created only by routers.

Note:

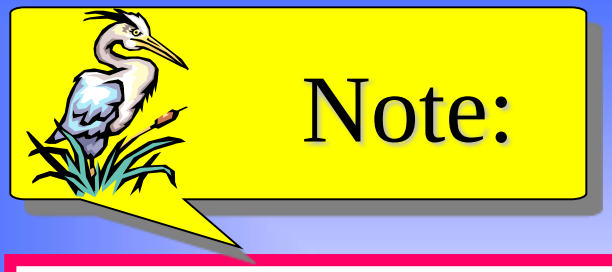
A router cannot detect all problems that prevent the delivery of a packet.

Note:

There is no flow-control mechanism in the IP protocol.

Figure 9.8 *Source-quench format*

Type: 4	Code: 0	Checksum
Unused (All 0s)		
Part of the received IP datagram including IP header plus the first 8 bytes of datagram data		



A source-quench message informs the source that a datagram has been discarded due to congestion in a router or the destination host.

The source must slow down the sending of datagrams until the congestion is relieved.

One source-quench message is sent for each datagram that is discarded due to congestion.

Whenever a router decrements a datagram with a time-to-live value to zero, it discards the datagram and sends a time-exceeded message to the original source.

When the final destination does not receive all of the fragments in a set time, it discards the received fragments and sends a time-exceeded message to the original source.

In a time-exceeded message, code 0 is used only by routers to show that the value of the time-to-live field is zero. Code 1 is used only by the destination host to show that not all of the fragments have arrived within a set time.

Figure 9.9 *Time-exceeded message format*

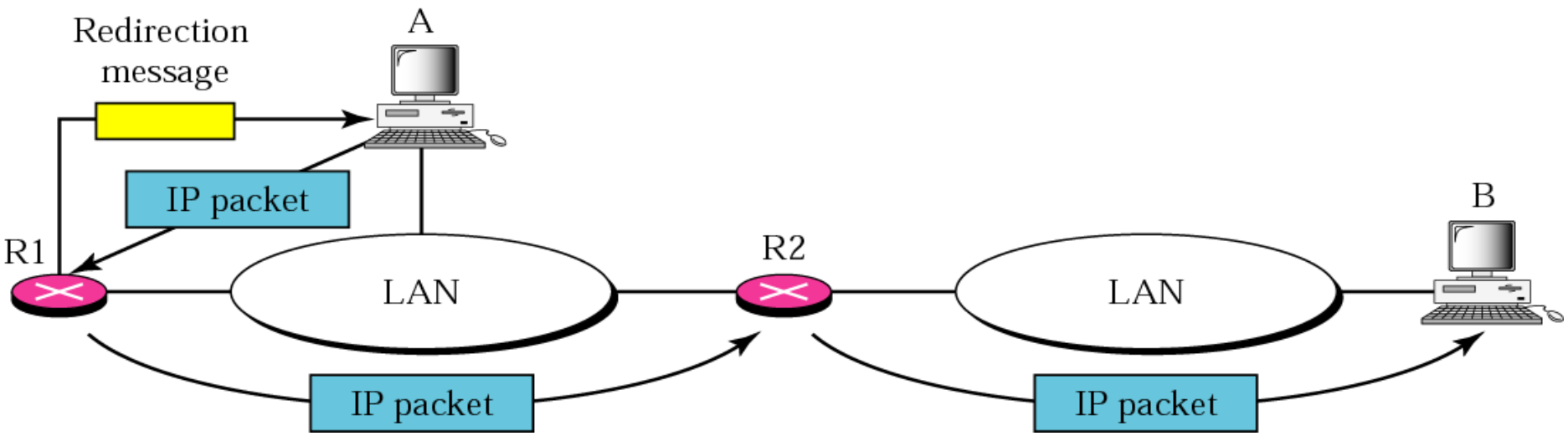
Type: 11	Code: 0 or 1	Checksum
Unused (All 0s)		
Part of the received IP datagram including IP header plus the first 8 bytes of datagram data		

A parameter-problem message can be created by a router or the destination host.

Figure 9.10 *Parameter-problem message format*

Type: 12	Code: 0 or 1	Checksum
Pointer	Unused (All 0s)	
Part of the received IP datagram including IP header plus the first 8 bytes of datagram data		

Figure 9.11 *Redirection concept*



A host usually starts with a small routing table that is gradually augmented and updated. One of the tools to accomplish this is the redirection message.

Figure 9.12 *Redirection message format*

Type: 5	Code: 0 to 3	Checksum
IP address of the target router		
Part of the received IP datagram including IP header plus the first 8 bytes of datagram data		

A redirection message is sent from a router to a host on the same local network.

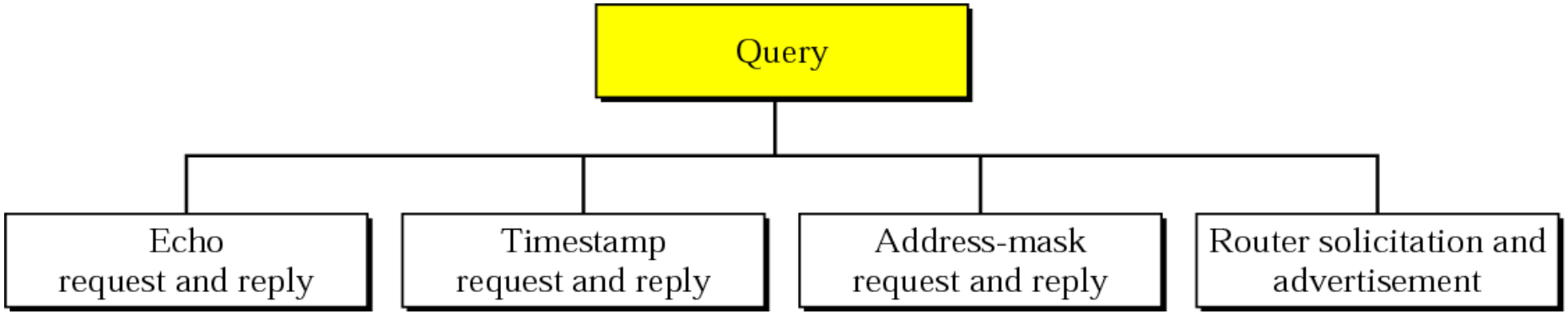
9.4 QUERY

ICMP can also diagnose some network problems through the query messages, a group of four different pairs of messages. In this type of ICMP message, a node sends a message that is answered in a specific format by the destination node.

The topics discussed in this section include:

Echo Request and Reply
Timestamp Request and Reply
Address-Mask Request and Reply
Router Solicitation and Advertisement

Figure 9.13 *Query messages*



An echo-request message can be sent by a host or router. An echo-reply message is sent by the host or router which receives an echo-request message.

Echo-request and echo-reply messages can be used by network managers to check the operation of the IP protocol.

Echo-request and echo-reply messages can test the reachability of a host. This is usually done by invoking the ping command.

Figure 9.14 *Echo-request and echo-reply messages*

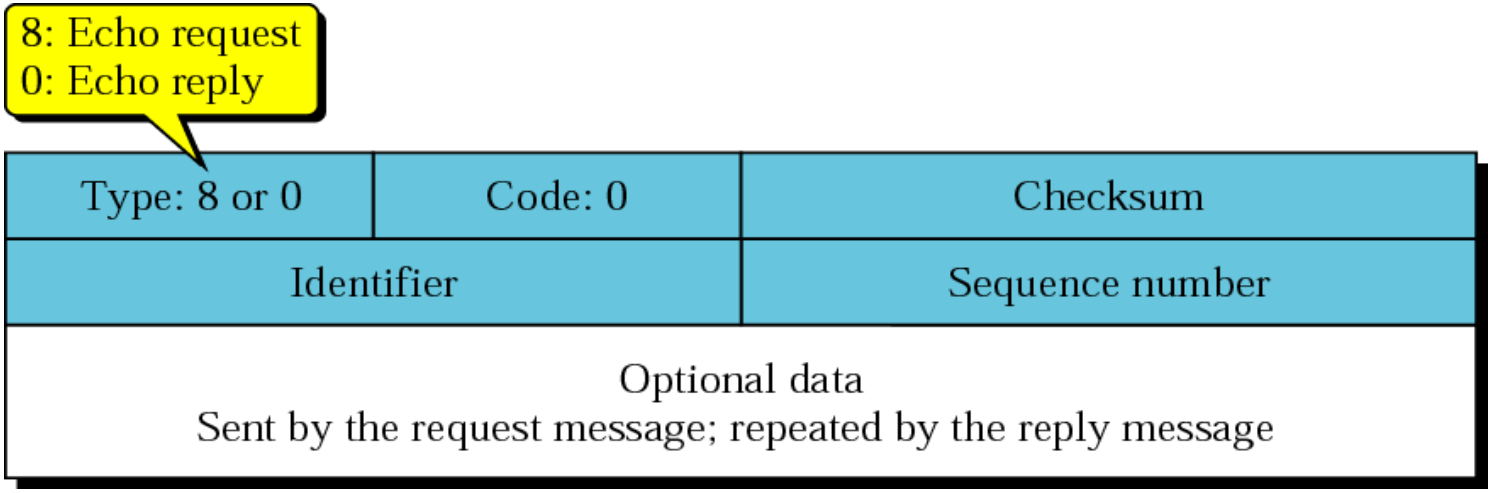
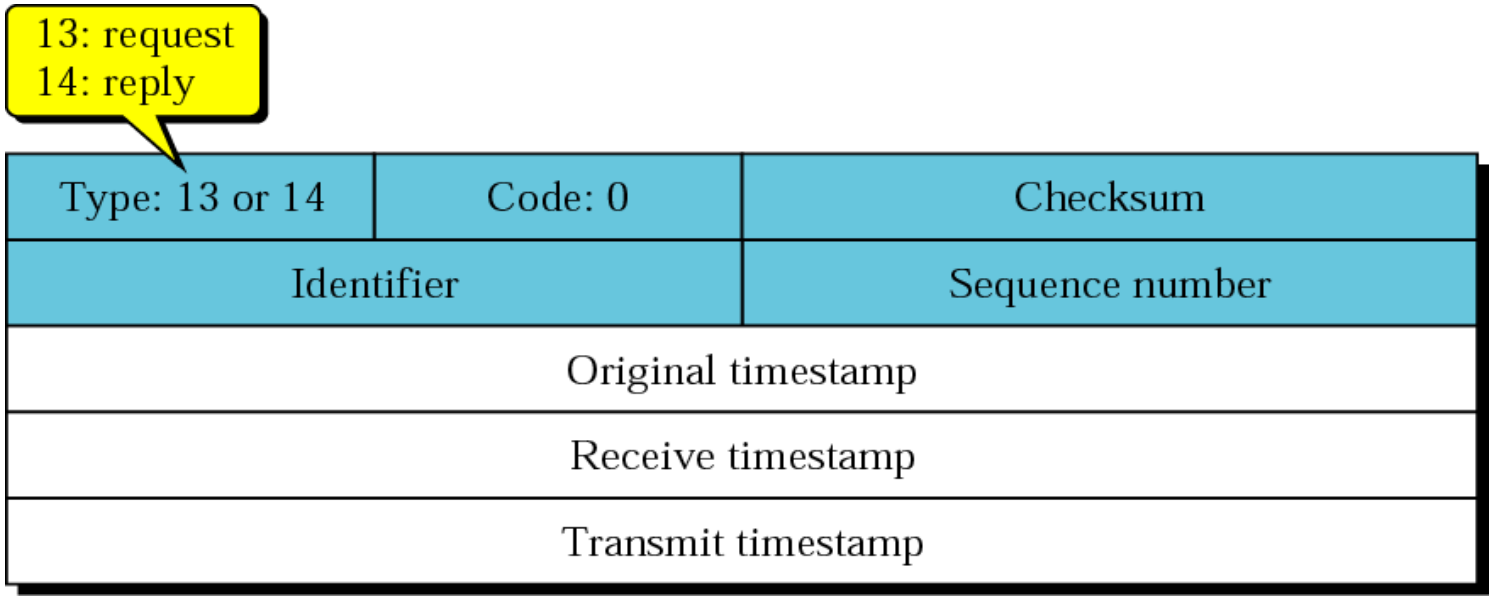


Figure 9.15 *Timestamp-request and timestamp-reply message format*



Timestamp-request and timestamp-reply messages can be used to calculate the round-trip time between a source and a destination machine even if their clocks are not synchronized.

The timestamp-request and timestamp-reply messages can be used to synchronize two clocks in two machines if the exact one-way time duration is known.

Figure 9.16 *Mask-request and mask-reply message format*

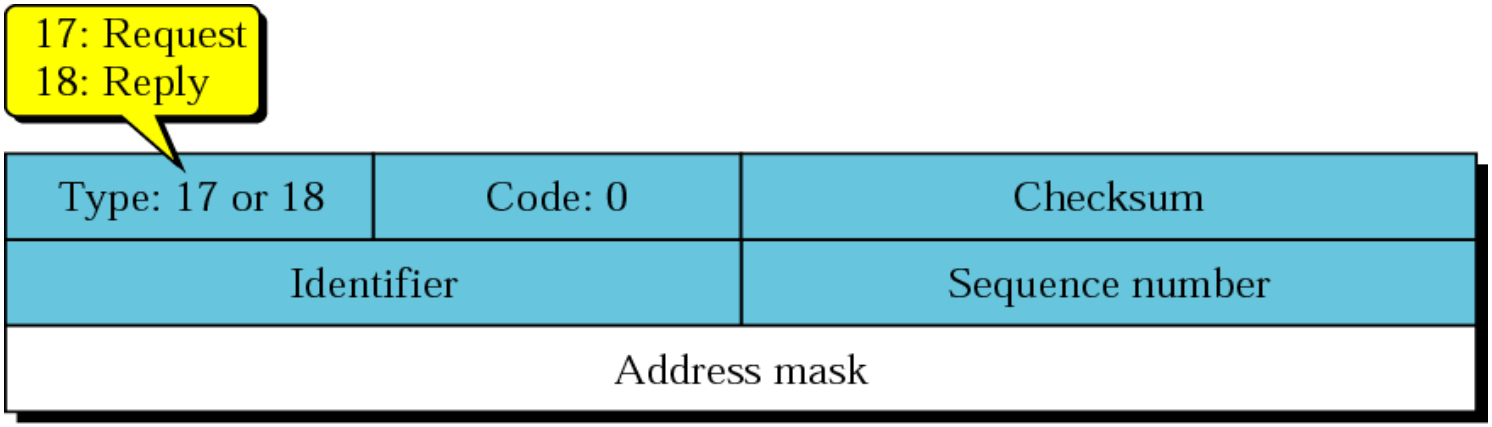


Figure 9.17 *Router-solicitation message format*

Type: 10	Code: 0	Checksum
Identifier		Sequence number

Figure 9.18 *Router-advertisement message format*

Type: 9	Code: 0	Checksum
Number of addresses	Address entry size	Lifetime
Router address 1		
Address preference 1		
Router address 2		
Address preference 2		
⋮		

9.5 CHECKSUM

In ICMP the checksum is calculated over the entire message (header and data).

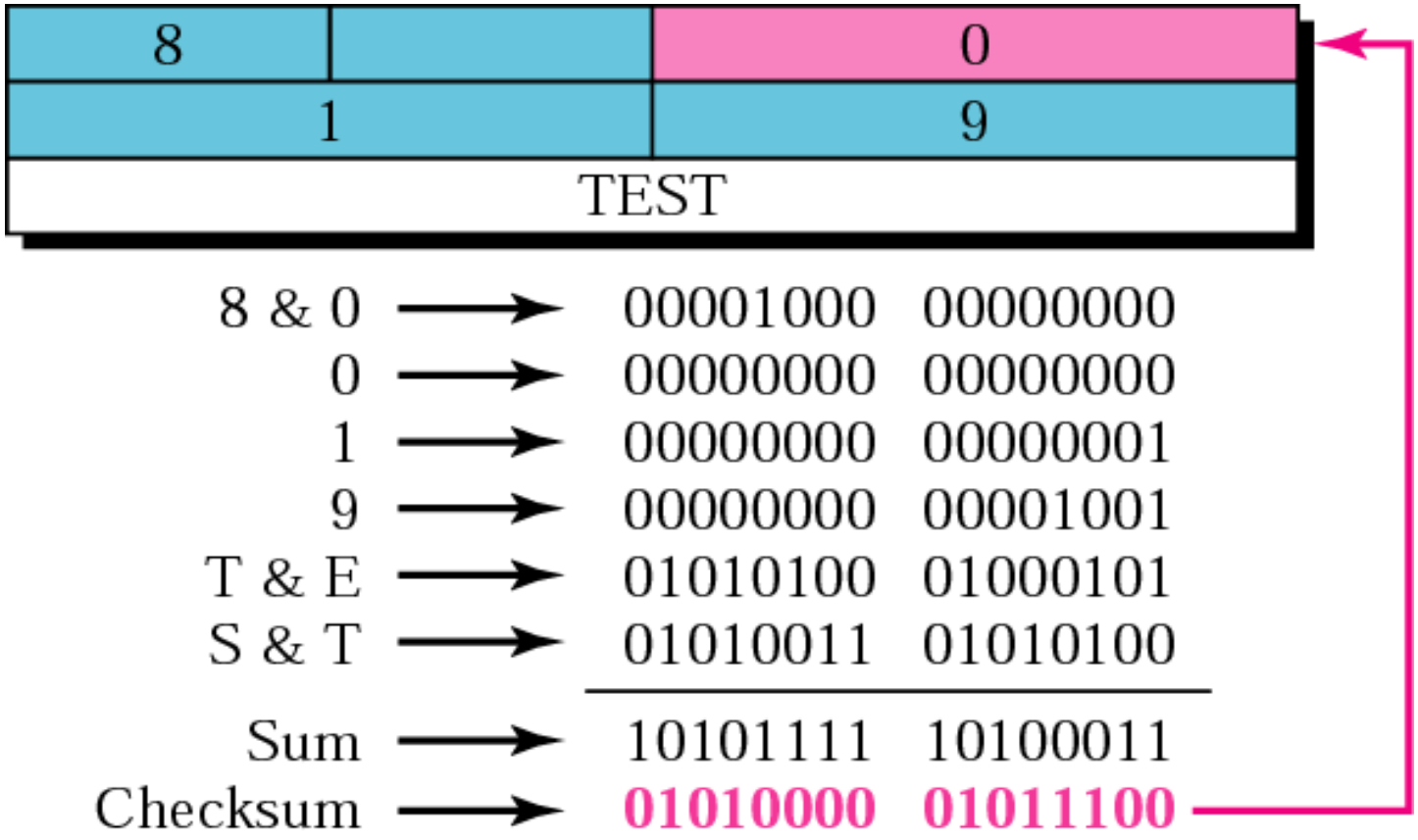
The topics discussed in this section include:

Checksum Calculation
Checksum Testing

EXAMPLE 1

Figure 9.19 shows an example of checksum calculation for a simple echo-request message (see Figure 9.14). We randomly chose the identifier to be 1 and the sequence number to be 9. The message is divided into 16-bit (2-byte) words. The words are added together and the sum is complemented. Now the sender can put this value in the checksum field.

Figure 9.19 *Example of checksum calculation*



9.6 DEBUGGING TOOLS

We introduce two tools that use ICMP for debugging: ping and traceroute.

The topics discussed in this section include:

Ping
Traceroute

EXAMPLE 2

We use the ping program to test the server fhda.edu. The result is shown below:

```
$ ping fhda.edu
PING fhda.edu (153.18.8.1) 56 (84) bytes of data.
64 bytes from tiptoe.fhda.edu (153.18.8.1): icmp_seq=0 ttl=62 time=1.91 ms
64 bytes from tiptoe.fhda.edu (153.18.8.1): icmp_seq=1 ttl=62 time=2.04 ms
64 bytes from tiptoe.fhda.edu (153.18.8.1): icmp_seq=2 ttl=62 time=1.90 ms
64 bytes from tiptoe.fhda.edu (153.18.8.1): icmp_seq=3 ttl=62 time=1.97 ms
64 bytes from tiptoe.fhda.edu (153.18.8.1): icmp_seq=4 ttl=62 time=1.93 ms
```

EXAMPLE 2 (CONTINUED)

```
64 bytes from tiptoe.fhda.edu (153.18.8.1): icmp_seq=5 ttl=62 time=2.00 ms
64 bytes from tiptoe.fhda.edu (153.18.8.1): icmp_seq=6 ttl=62 time=1.94 ms
64 bytes from tiptoe.fhda.edu (153.18.8.1): icmp_seq=7 ttl=62 time=1.94 ms
64 bytes from tiptoe.fhda.edu (153.18.8.1): icmp_seq=8 ttl=62 time=1.97 ms
64 bytes from tiptoe.fhda.edu (153.18.8.1): icmp_seq=9 ttl=62 time=1.89 ms
64 bytes from tiptoe.fhda.edu (153.18.8.1): icmp_seq=10 ttl=62 time=1.98 ms

--- fhda.edu ping statistics ---
11 packets transmitted, 11 received, 0% packet loss, time 10103ms

rtt min/avg/max = 1.899/1.955/2.041 ms
```

EXAMPLE 3

For the this example, we want to know if the adelphia.net mail server is alive and running. The result is shown below:

```
$ ping mail.adelphia.net
PING mail.adelphia.net (68.168.78.100) 56(84) bytes of data.
64 bytes from mail.adelphia.net (68.168.78.100): icmp_seq=0 ttl=48 time=85.4 ms
64 bytes from mail.adelphia.net (68.168.78.100): icmp_seq=1 ttl=48 time=84.6 ms
64 bytes from mail.adelphia.net (68.168.78.100): icmp_seq=2 ttl=48 time=84.9 ms
64 bytes from mail.adelphia.net (68.168.78.100): icmp_seq=3 ttl=48 time=84.3 ms
64 bytes from mail.adelphia.net (68.168.78.100): icmp_seq=4 ttl=48 time=84.5 ms
```

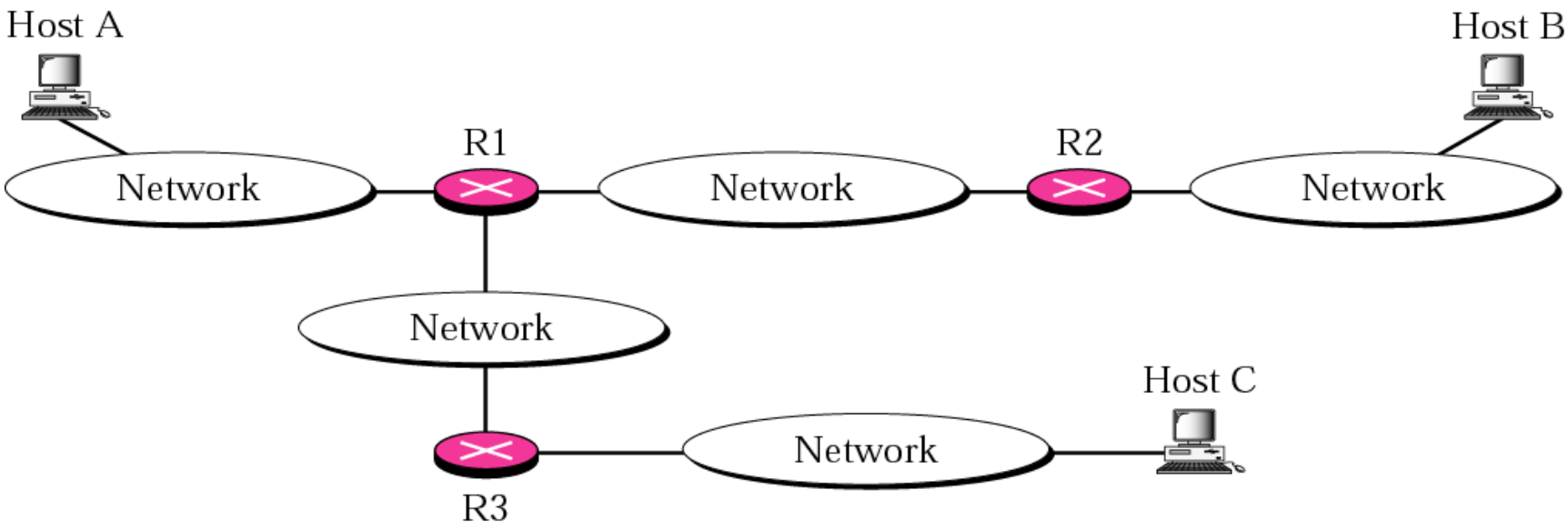
EXAMPLE 3 (CONTINUED)

```
64 bytes from mail.adelphia.net (68.168.78.100): icmp_seq=5 ttl=48 time=84.7 ms
64 bytes from mail.adelphia.net (68.168.78.100): icmp_seq=6 ttl=48 time=84.6 ms
64 bytes from mail.adelphia.net (68.168.78.100): icmp_seq=7 ttl=48 time=84.7 ms
64 bytes from mail.adelphia.net (68.168.78.100): icmp_seq=8 ttl=48 time=84.4 ms
64 bytes from mail.adelphia.net (68.168.78.100): icmp_seq=9 ttl=48 time=84.2 ms
64 bytes from mail.adelphia.net (68.168.78.100): icmp_seq=10 ttl=48 time=84.9 ms
64 bytes from mail.adelphia.net (68.168.78.100): icmp_seq=11 ttl=48 time=84.6 ms
64 bytes from mail.adelphia.net (68.168.78.100): icmp_seq=12 ttl=48 time=84.5 ms

--- mail.adelphia.net ping statistics ---

14 packets transmitted, 13 received, 7% packet loss, time 13129ms
rtt min/avg/max/mdev = 84.207/84.694/85.469
```

Figure 9.20 *The traceroute program operation*



EXAMPLE 4

We use the traceroute program to find the route from the computer voyager.deanza.edu to the server fhda.edu. The following shows the result:

```
$ traceroute fhda.edu
traceroute to fhda.edu (153.18.8.1), 30 hops max, 38 byte packets
1 Dcore.fhda.edu (153.18.31.254) 0.995 ms 0.899 ms 0.878 ms
2 Dbackup.fhda.edu (153.18.251.4) 1.039 ms 1.064 ms 1.083 ms
3 tiptoe.fhda.edu (153.18.8.1) 1.797 ms 1.642 ms 1.757 ms
```

EXAMPLE 4 (CONTINUED)

The un-numbered line after the command shows that the destination is 153.18.8.1. The TTL value is 30 hops. The packet contains 38 bytes: 20 bytes of IP header, 8 bytes of UDP header, and 10 bytes of application data. The application data is used by traceroute to keep track of the packets.

The first line shows the first router visited. The router is named Dcore.fhda.edu with IP address 153.18.31.254. The first round trip time was 0.995 milliseconds, the second was 0.899 milliseconds, and the third was 0.878 milliseconds.

The second line shows the second router visited. The router is named Dbackup.fhda.edu with IP address 153.18.251.4. The three round trip times are also shown.

The third line shows the destination host. We know that this is the destination host because there are no more lines. The destination host is the server fhda.edu, but it is named tiptoe. fhda.edu with the IP address 153.18.8.1. The three round trip times are also shown.

EXAMPLE 5

In this example, we trace a longer route, the route to xerox.com

```
$ traceroute xerox.com
traceroute to xerox.com (13.1.64.93), 30 hops max, 38 byte packets
1 Dcore.fhda.edu (153.18.31.254) 0.622 ms 0.891 ms 0.875 ms
2 Ddmz.fhda.edu (153.18.251.40) 2.132 ms 2.266 ms 2.094 ms
...

18 alpha.Xerox.COM (13.1.64.93) 11.172 ms 11.048 ms 10.922 ms
```

Here there are 17 hops between source and destination. Note that some round trip times look unusual. It could be that a router is too busy to process the packet immediately.

EXAMPLE 6

An interesting point is that a host can send a traceroute packet to itself. This can be done by specifying the host as the destination. The packet goes to the loopback address as we expect.

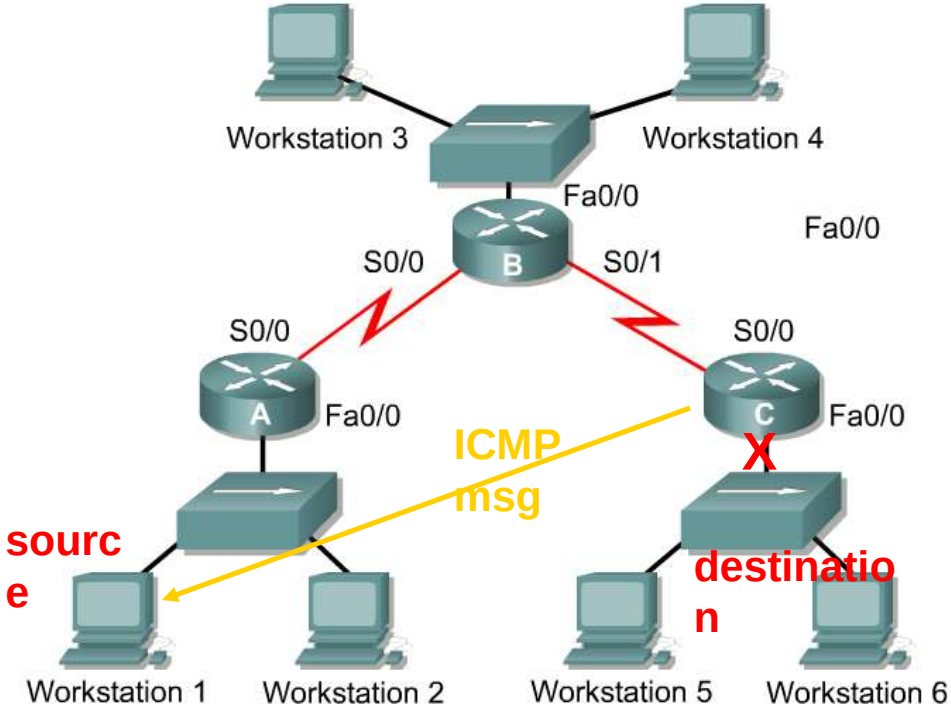
```
$ traceroute voyager.deanza.edu
traceroute to voyager.deanza.edu (127.0.0.1), 30 hops max, 38 byte packets
1 voyager (127.0.0.1) 0.178 ms 0.086 ms 0.055 ms
```

EXAMPLE 7

Finally, we use the traceroute program to find the route between fhda.edu and mhhe.com (McGraw-Hill server). We notice that we cannot find the whole route. When traceroute does not receive a response within 5 seconds, it prints an asterisk to signify a problem, and then tries the next hop..

```
$ traceroute mhhe.com
traceroute to mhhe.com (198.45.24.104), 30 hops max, 38 byte packets
1 Dcore.fhda.edu (153.18.31.254) 1.025 ms 0.892 ms 0.880 ms
2 Ddmz.fhda.edu (153.18.251.40) 2.141 ms 2.159 ms 2.103 ms
3 Cinic.fhda.edu (153.18.253.126) 2.159 ms 2.050 ms 1.992 ms
   ...
16 * * *
17 * * *
...............
```

Error reporting and error correction



- **When datagram delivery errors occur, ICMP is used to report these errors back to the source of the datagram.**
- **Example**

- Workstation 1 is sending a datagram to Workstation 6
- Fa0/0 on Router C goes down
- Router C then utilizes ICMP to send a message back to Workstation 1 indicating that the datagram could not be delivered.
- ICMP does not correct the encountered network problem.
- Router C knows only the source and destination IP addresses of the datagram, not know about the exact path the datagram took to Router C, therefore, Router C can only notify Workstation 1 of the failure
- ICMP reports on the status of the delivered packet only to the source device.

9.7 ICMP PACKAGE

To give an idea of how ICMP can handle the sending and receiving of ICMP messages, we present our version of an ICMP package made of two modules: an input module and an output module.

The topics discussed in this section include:

Input Module
Output Module

Figure 9.21 *ICMP package*

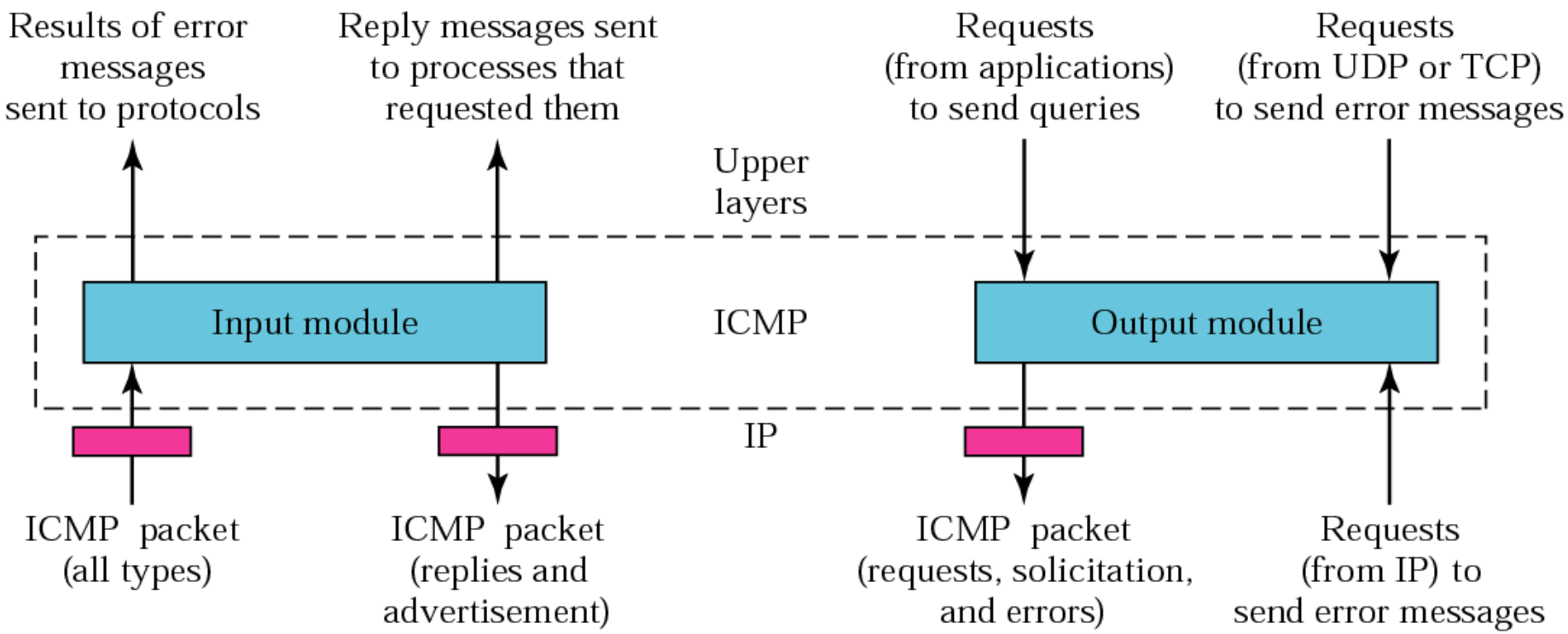


Table 9.2 *Input Module*

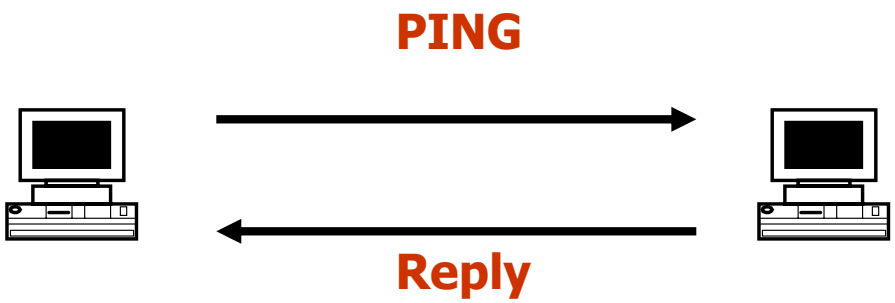
```
ICMP_Input_module (ICMP_Packet)
{
    If (the type is a request)
    {
         Create a reply
         Send the reply
    }
    If (the type defines a redirection)
    {
        Modify the routing table
    }
    If (the type defines other error messages)
    {
        Inform the appropriate source protocol
    }
    Return
}
```

Table 9.3 *Output Module*

```
ICMP_Output_Module (demand)
{
    If (the demand defines an error message)
    {
            If (demand comes from IP AND is forbidden)
            {
                 Return
            }
            If (demand is a valid redirection message)
            {
                 Return
            }
            Create an error message
    If (demand defines a request)
    {
         Create a request message
    }
    Send the message
    Return
}
```

PING : ICMP Echo Request/Reply

- PING sends and ICMP echo request to a remote host, which then return an ICMP echo reply to the sender
- All TCP/IP node is supposed to implement ICMP and respond to ICMP echo



PING Command

- Send a single echo request message and wait for a reply
- Another request is sent if the reply is not received within one second
- Continue until at least one reply is received or stop after time out

> ping maliwan

- maliwan.psu.ac.th is alive

If maliwan down

>ping maliwan

- no answer from maliwan.psu.ac.th

PING Command

- Send an echo request message every seconds and records the time it takes for each reply
- every echo request contains a unique sequence number to match reply and request
- also record round-trip timing
- also do packet lost statistics

PING Example

```
C:\>ping maliwan.psu.ac.th

Pinging maliwan.psu.ac.th [192.168.100.4] with 32 bytes of data:

Reply from 192.168.100.4: bytes=32 time=3ms TTL=32
Reply from 192.168.100.4: bytes=32 time=3ms TTL=32
Reply from 192.168.100.4: bytes=32 time=3ms TTL=32
Reply from 192.168.100.4: bytes=32 time=4ms TTL=32

Ping statistics for 192.168.100.4:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 3ms, Maximum = 4ms, Average = 3ms

C:\>_
```

What we get from PING?

- Timing information
- Connection reliability
- Destination Unreachable (routable)

PING Results

- no respond

  no end node, no connection
- lost packet (significant when > 2-3 %)

  transmission error on LAN/WAN, overloading bridge or router
- time acknowledge vary

  host/network overloading > 100 ms make telnet less acceptable
- no lost and echo time is reasonably constant

Traceroute Command

- Command to determine the active route to a destination address
- How?
  - Send a UDP message to an unused port on the target host with ttl = 1
  - router decrease ttl to 0, it has to return an ICMP time exceed massage
  - traceroute set ttl = 2 and retransmits, this time go one more hop
  - ttl++ until UDP reach the destination
  - the target returns an ICMP service unreachable because there is no UDP port service

Traceroute Example

```
C:\>tracert www.psu.ac.th
Tracing route to s1.psu.ac.th [192.168.100.61]
over a maximum of 30 hops:
  1    1 ms    1 ms    1 ms  cs-gw.cs.psu.ac.th [172.28.80.1]
  2    2 ms    2 ms    1 ms  esw-cc.psu.ac.th [192.168.99.39]
  3    2 ms    3 ms    5 ms  cc-atm.psu.ac.th [192.168.0.249]
  4    4 ms    2 ms    3 ms  tooky.psu.ac.th [192.168.98.11]
  5    3 ms    3 ms    3 ms  s1.psu.ac.th [192.168.100.61]
Trace complete.
C:\>
```

- usually probes each hop 3 times
- a lost message or a router that doesn't respond with denote with an " * "

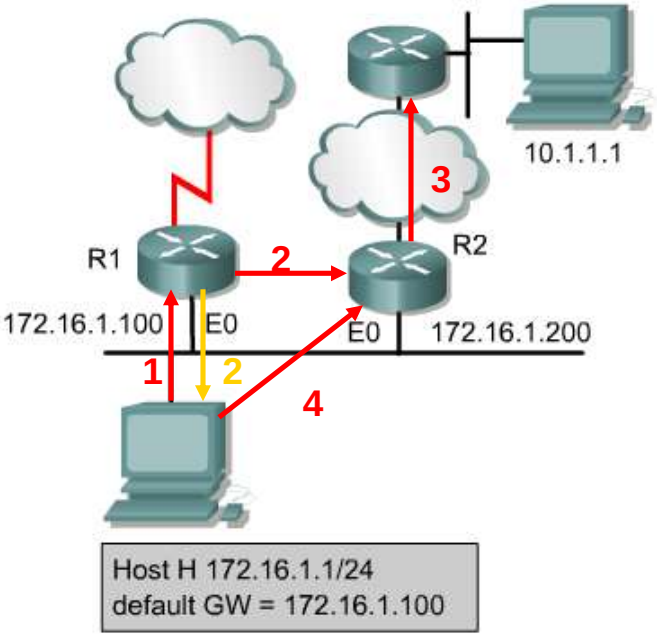
Introduction to ICMP Control Messages

- Unlike error messages, control messages are <u>not</u> the results of lost packets or error conditions which occur during packet transmission.
- Instead, they are used to inform hosts of conditions such as:
  - Network congestion
  - Existence of a better gateway to a remote network

ICMP Redirect

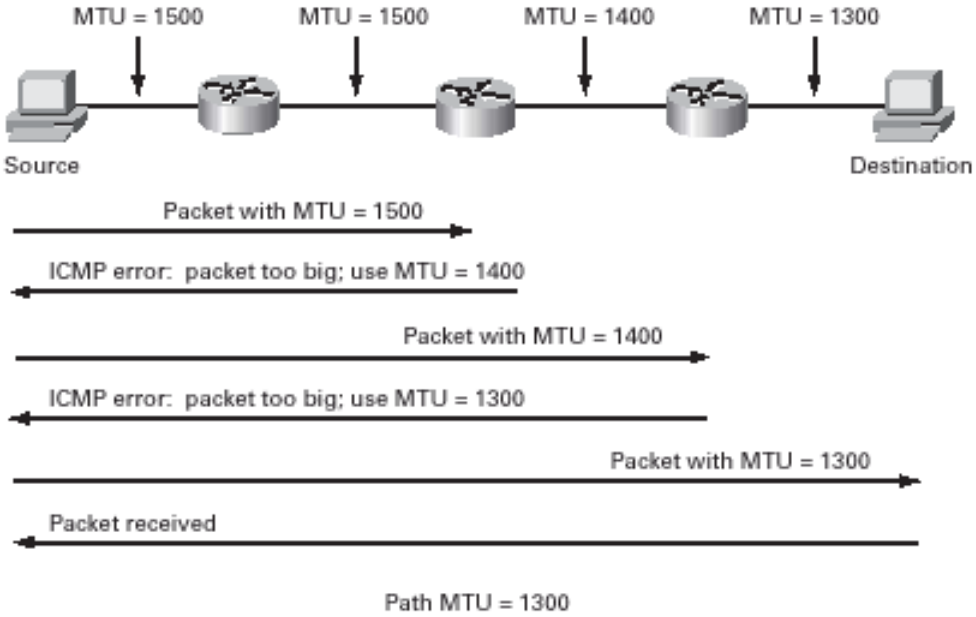
ICMP Redirect

Type = 5 Code = 0 to 3



- ICMP Redirect messages can only be sent by routers
- Host H sends a packet to Host 10.1.1.1 on network 10.0.0.0/8.
- Since Host H is not directly connected to the same network, it forwards the packet to its default gateway, Router R1 at 172.16.1.100.
- Router R1 finds the correct route to network 10.0.0.0/8 by looking in its route table.
- It determines that the path to the network is back out the same interface the request to forward the packet came from to Router R2 at 172.16.1.200.
- R1 forwards the packet to **R2 and sends an ICMP redirect/change request to Host H** telling it to use Router R2 at 172.16.1.100 as the gateway to forward all future requests to network 10.0.0.0/8.

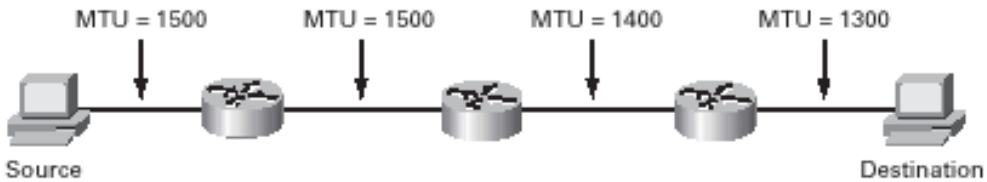
Path MTU Discovery



Problem:

- How path MTU discovery (PMTU-D) combined with filtering ICMP messages can result in connectivity problems.
- Path MTU discovery allows a node to dynamically discover and adjust to differences in the MTU size of every link along a given data path.
- In IPv4, the minimum link MTU size is 68 octets and the recommended minimum is 576 octets, which is the minimum reassembly buffer size.
- So, any IPv4 packet must be at least 68 octets in length.
- (*In IPv6, the minimum link MTU is 1280 octets, but the recommended MTU value for IPv6 links is 1500 octets. The maximum packet size supported by the basic IPv6 header is 64,000 octets. Larger packets called jumbograms could be handled using a hop-by-hop extension header option*.)

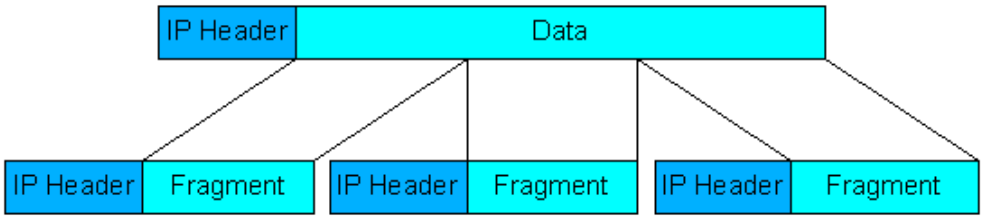
Path MTU Discovery - Terms

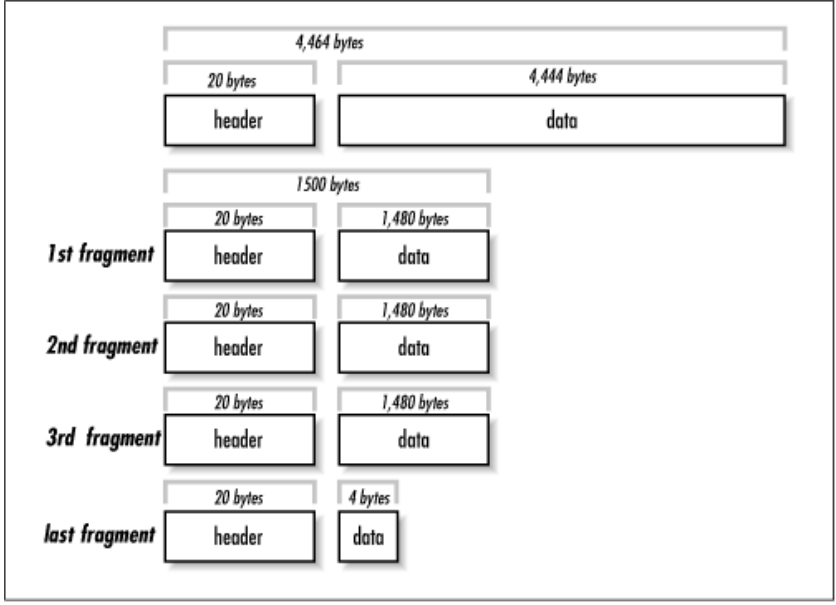


Topology	MTU (in bytes)	Defined By
Hyperchannel	65,535	RFC 1374
16 MB/s Token Ring	17,914	IBM
802.4 Token Bus	8,166	RFC 1042
4 MBs Token Ring	4,464	RFC 1042
FDDI	4,352	RFC 1390
DIX Ethernet	1,500	RFC 894
Point-to-Point Protocol (PPP)	1,500	RFC 1548
802.3 Ethernet	1,492	RFC 1042
Serial-Line IP (SLIP)	1,006	RFC 1055
X.25 & ISDN	576	RFC 1356
ARCnet	508	RFC 1051

- **MTU**: The maximum transmission unit is a link layer restriction on the maximum number of bytes of data in a single transmission (ie. frame, cell, packet, depending on the terminology).
  - The table above shows some typical values for MTUs, taken from RFC-1191.
- **Path MTU**: The smallest MTU of any link on the current path between two hosts.
  - This may change over time since the route between two hosts, especially on the Internet, may change over time.
  - It is not necessarily symmetric and can even vary for different types of traffic from the same host.

Terms



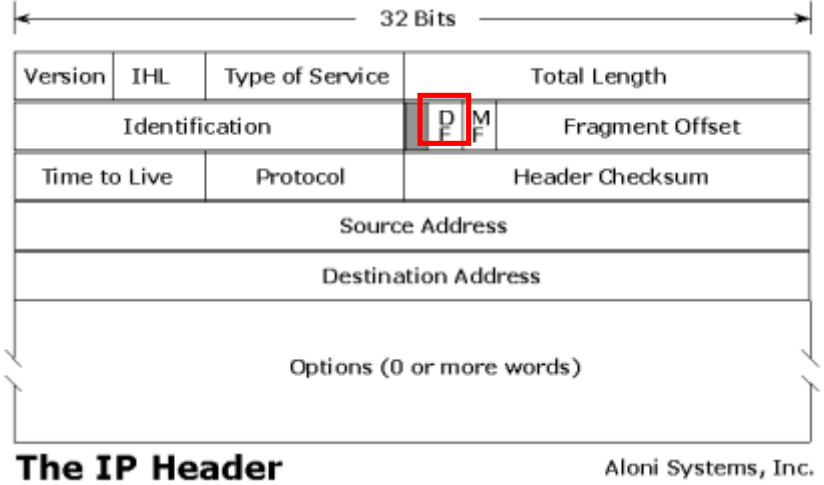


- **Fragmentation**: When a packet is too large to be sent across a link as a single unit, a router can fragment the packet.
  - This means that it splits it into multiple parts which contain enough information for the receiver to glue them together again.
  - Note that this is <u>not</u> done on a hop-by-hop basis, but once fragmented a packet will not be put back together until it reaches its destination.
  - Fragmentation is undesirable for numerous reasons, including:
    - If any one fragment from a packet is dropped, the entire packet needs to be retransmitted. This is a very significant problem.
    - It imposes extra processing load on the routers that have to split the packets.
    - In some configuration, simpler firewalls will block all fragments because they don't contain the header information for a higher layer protocol (eg. TCP) needed for filtering.

Terms

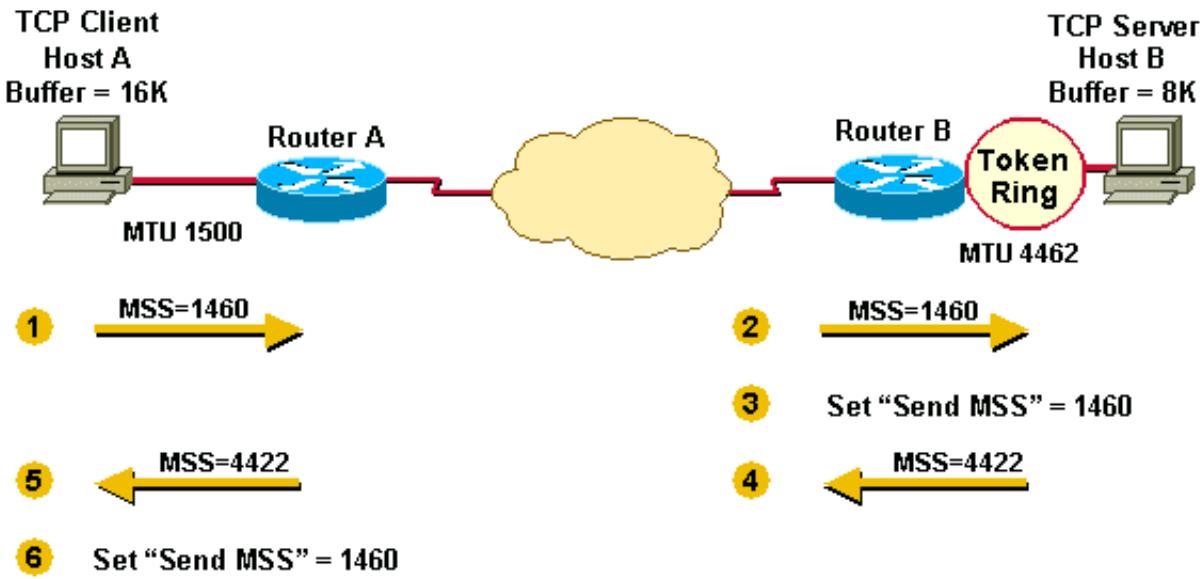
3 Type (8 bits)	4 Code (8 bits)	Checksum (16 bits)
Parameters		

ICMP Destination Unreachable
Fragmentation needed, but DF Set



- **DF (Don't Fragment) bit**: This is a bit in the IP header that can be set to indicate that the packet should not be fragmented by routers.
  - If the packet needs to be fragmented, an *ICMP "can't fragment"* error is returned sent to the sender and the packet is dropped.
- **ICMP Can't Fragment Error**:
  - This error is a type 3 (destination unreachable), code 4 (fragmentation needed but don't-fragment bit set)
  - Returned by a router when it receives a packet that is too large for it to forward and the DF bit is set.
  - The packet is dropped and the ICMP error is sent back to the origin host.
  - Normally, this tells the origin host that it needs to reduce the size of its packets if it wants to get through.
  - Recent systems also include the MTU of the next hop in the ICMP message so the source knows how big its packets can be.
  - Note that this error is only sent if the DF bit is set; otherwise, packets are just fragmented and passed through.

Terms

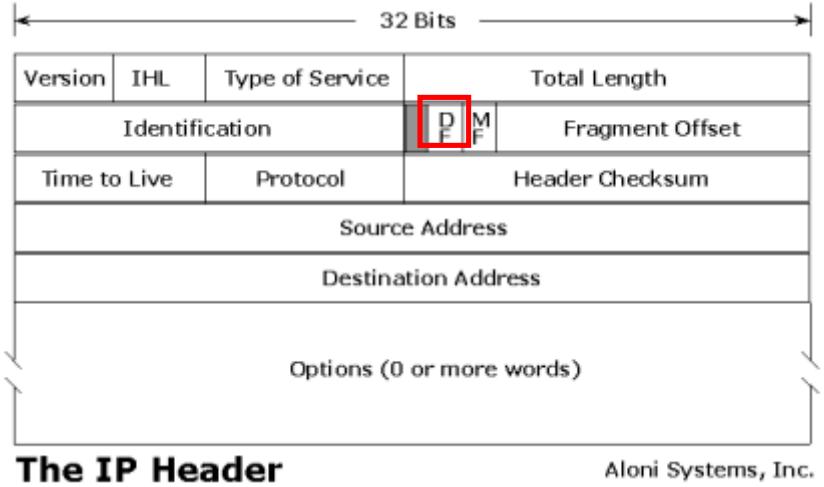


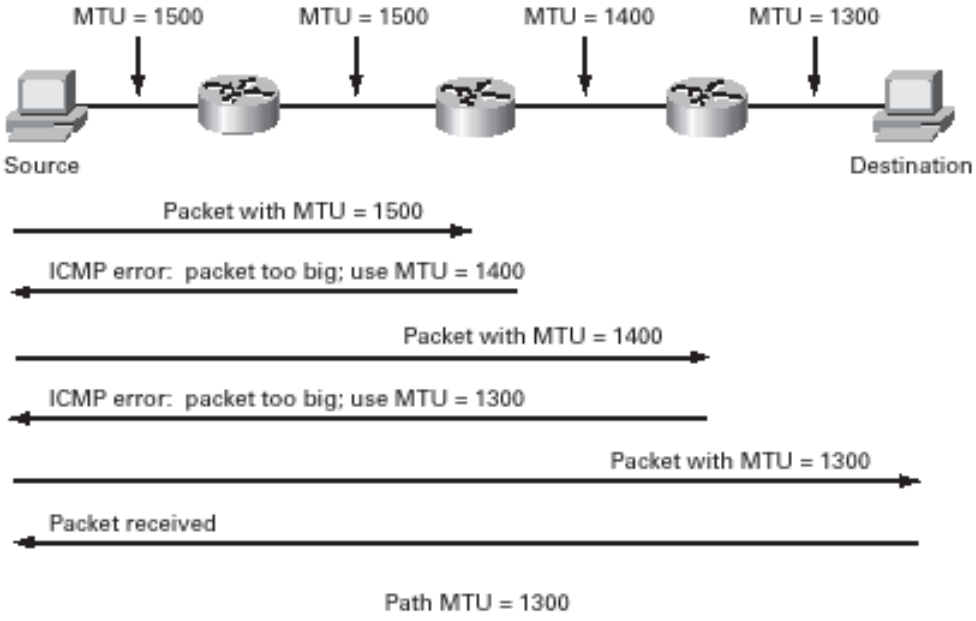
- MSS: The MSS is the maximum segment size.
  - It can be announced during the establishment of a TCP connection to indicate to the other end the largest amount of data in one packet that should be sent by the remote system.
  - MSS is beyond the scope of this discussion.

Path MTU Discovery (PMTU-D)

- Now you know that Path MTUs vary.
- You know that fragmentation is bad.
- The solution?
  - Well, one solution is Path MTU Discovery.
  - The idea behind it is to send packets that are as large as possible while still avoiding fragmentation.

PMTU-D

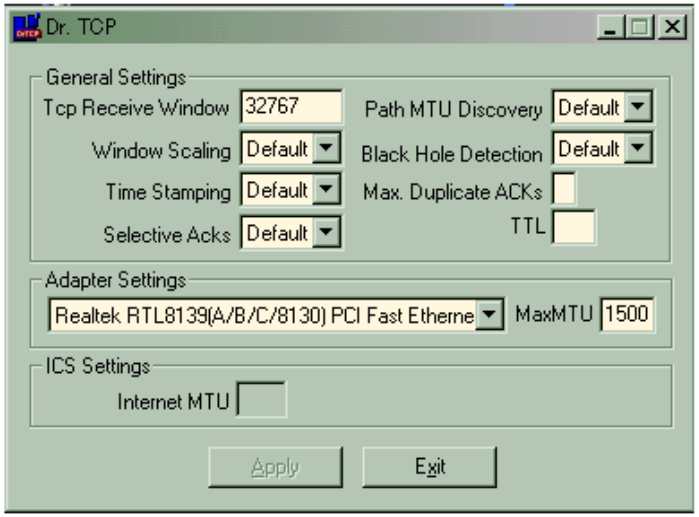


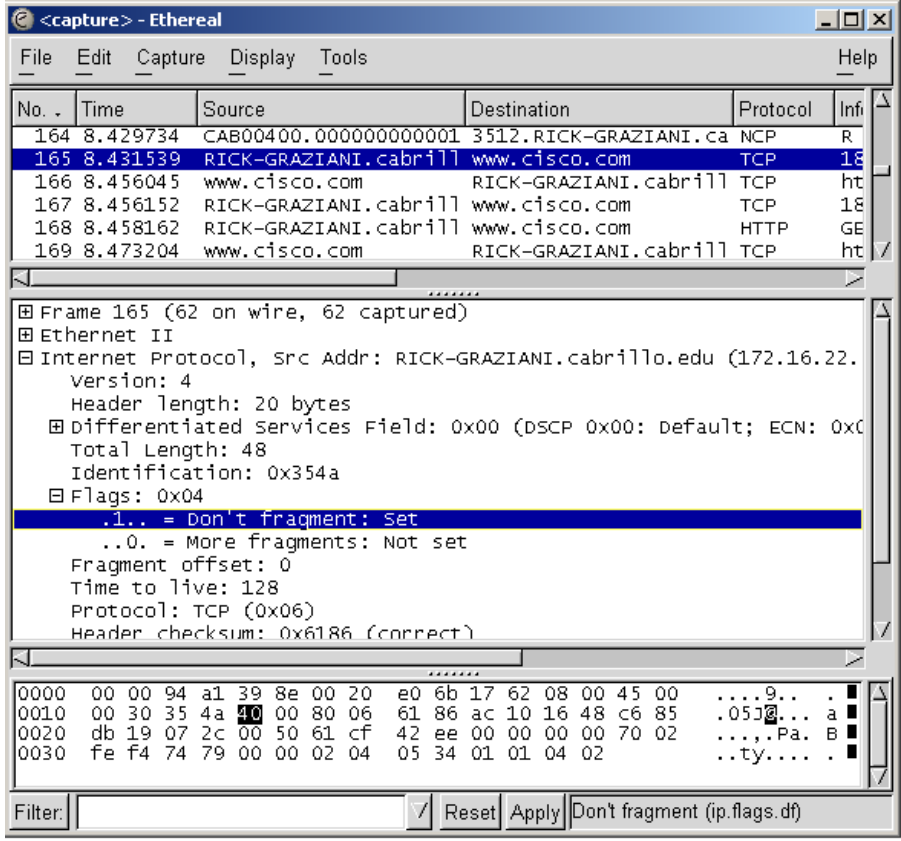


- A host does this by starting by sending packets that have a max. size of the lesser of the local MTU or the MSS announced by the remote system.
- These packets are sent with the **DF bit set**.
- If there is some MTU between the two hosts which is too small to pass the packet successfully, then an ICMP can't fragment error will be sent back to the source.
- It will then know to lower the size; if the ICMP message includes the next hop MTU, it can pick the correct size for that link immediately, otherwise it has to guess.

PMTU-D

http://www.dslreports.com





- The exact process that systems go through is somewhat more complicated to account for special circumstances. See, RFC 1191.
- A good indication of if a system is trying to do PMTU-D is to watch the packets it is sending with something like tcpdump or snoop and see if they have the DF bit set; if so, it is most likely trying to do PMTU-D.
- Most Windows and Linux/Unix OS's default to using PMTU-D.
- Adjusting IP MTU, TCP MSS, and PMTUD on Windows and Sun Systems - http://www.cisco.com/warp/public/105/38.shtml

The problem with ICMP filtering and PMTU-D

- Many network administrators have decided to filter ICMP at a router or firewall.
- There are valid (and many invalid) reasons for doing this, however it can cause problems.
- ICMP is an integral part of the Internet and can <u>not</u> be filtered without due consideration for the effects.
- In this case, if the ICMP can't fragment errors can not get back to the source host due to a filter, the host will never know that the packets it is sending are too large.
- This means it will keep trying to send the same large packet, and it will keep being dropped--silently dropped from the view of any system on the other side of the filter.
- While a small handful of systems that implement PMTU-D also implement a way to detect such situations, most don't and even for those that do it has a negative impact on performance and the network.

The Symptoms

```
Command Prompt

C:\>ping 198.133.219.2 -l 100

Pinging 198.133.219.2 with 100 bytes of data:

Reply from 198.133.219.2: bytes=100 time=15ms TTL=241
Reply from 198.133.219.2: bytes=100 time=13ms TTL=241
Reply from 198.133.219.2: bytes=100 time=19ms TTL=241
Reply from 198.133.219.2: bytes=100 time=13ms TTL=241

Ping statistics for 198.133.219.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 13ms, Maximum = 19ms, Average = 15ms

C:\>ping 198.133.219.2 -l 1100

Pinging 198.133.219.2 with 1100 bytes of data:

Reply from 198.133.219.2: bytes=1100 time=17ms TTL=241
Reply from 198.133.219.2: bytes=1100 time=18ms TTL=241
Reply from 198.133.219.2: bytes=1100 time=17ms TTL=241
Reply from 198.133.219.2: bytes=1100 time=17ms TTL=241

Ping statistics for 198.133.219.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 17ms, Maximum = 18ms, Average = 17ms

C:\>ping 198.133.219.2 -l 1500

Pinging 198.133.219.2 with 1500 bytes of data:

Request timed out.
Request timed out.
Request timed out.
Request timed out.

Ping statistics for 198.133.219.2:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),

C:\>_
```

- If this is happening, typical symptoms include the ability for small packets (eg. request a very small web page) to get through, but larger ones (eg. a large web page) will simply hang.
- This situation can be confusing to the novice administrator because they obviously have some connectivity to the host, but it just stops working for no obvious reason on certain transfers.

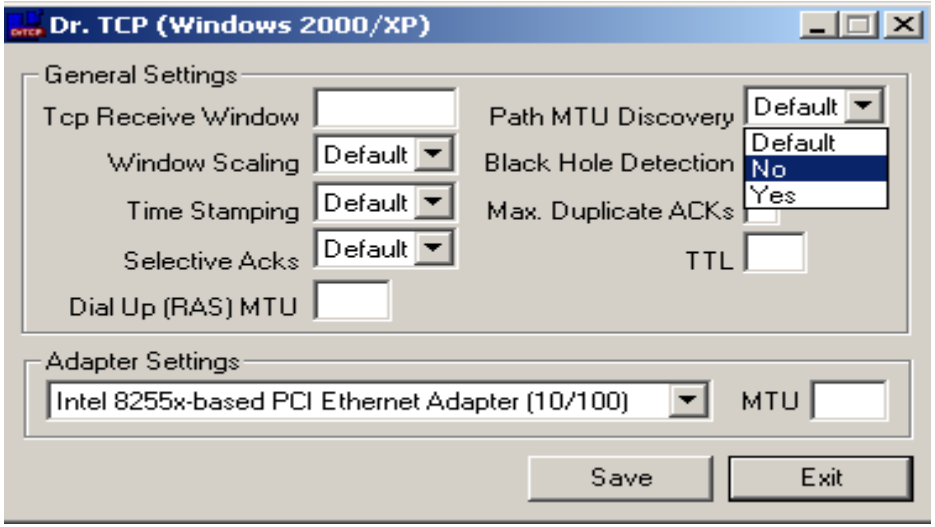
The Fix

- There is one solution, and several workarounds, for this problem.
- The Fix:

- Fix your filters!
  - The real problem here is filtering ICMP messages without understanding the consequences.
  - Many packet filters will allow you to setup filters to only allow certain types of ICMP messages through.
  - If you reconfigure them to let ICMP can't fragment (type 3, code 4) messages through, the problem should disappear.
  - If the filter is somewhere between you and the other end, contact the administrator of that machine and try to convince them to fix the problem.
- We will learn how to do this on Router Access Control Lists (ACLs)

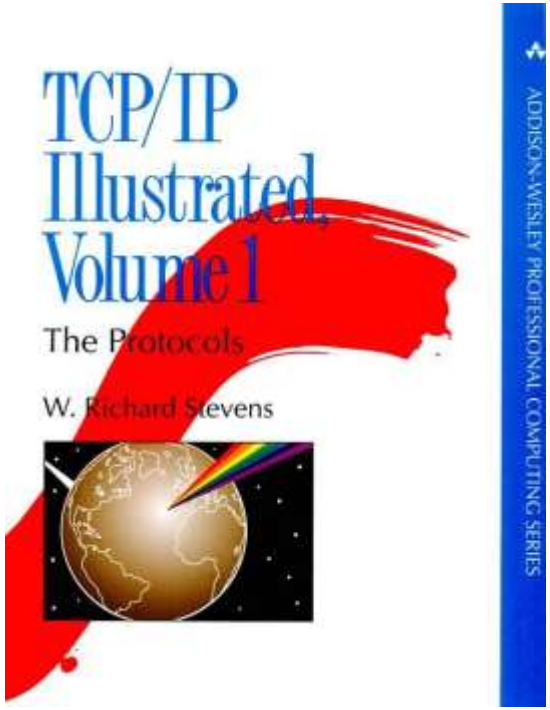
The Workarounds

- Reduce the MTU on the machines at one end or the other.
  - This is a workaround and should not be done unless necessary.
  - If you reduce the MTU on the system trying to do path MTU discovery to a point where it is less than or equal to the former path MTU, it will no longer try sending packets large enough to cause problems.
  - Similarly, if you change the MTU on the system on the other end, it will advertise a lower MSS so the sending system will only send packets with data that fits into that MSS.



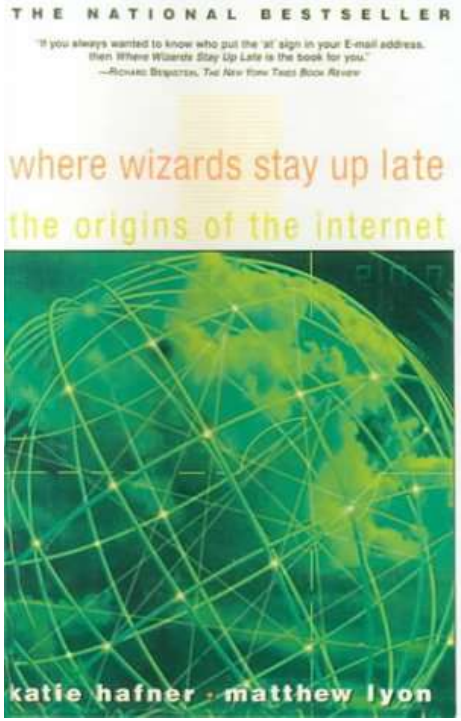
- Disable PMTU-D; if you control access to the machine that is trying to do PMTU-D, and are unable to get the person administering the bogus filter to fix it, disabling PMTU-D will fix the problem for data sent by that machine.
  - Data being received by the machine, however, can still run into the problem.
  - With the size that HTTP requests are growing to, this could start to be a problem more and more; historically, HTTP requests have nearly always been small enough to fit through links with small MTUs in one packet.
  - Disabling PMTU-D is simply a workaround, and should not generally be done unless necessary or you know what you are doing.

Recommended Reading



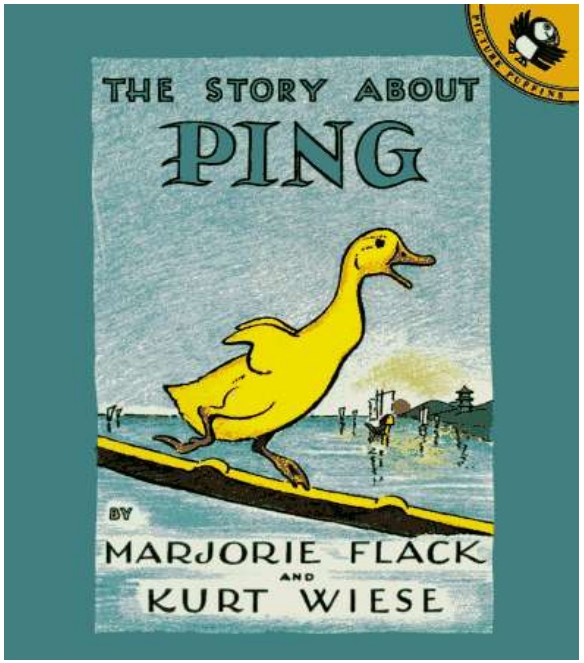
TCP/IP Illustrated, Vol. 1 W. Richard Stevens
Addison-Wesley Pub Co ISBN: 0201633469.

Although, published in 1994, written by the late Richard Stevens, it is still regarded as the definitive book on TCP/IP.



Where Wizards Stay Up Late
Katie Hafner and Matthew Lyon
ISBN 0613181530

For more information on Ping

Here are two options for more information on Ping:

- Read the book: **The Story About Ping**
  by Marjorie Flack, Kurt Wiese (See a Amazon.com customer review on next slide – very funny!